

PRODUCT REQUIREMENT DOCUMENT

3D Object Reconstruction from Images

A Graph CNN and Mesh Deformation based Single-Image 3D Reconstruction
System for E-Commerce and AR/VR

Track: Multimodal AI | Theme: 3D Vision

Industry Domain: E-Commerce / AR-VR

Duration: 3 Months

Grounding Paper: Pixel2Mesh: Generating 3D Mesh Models from Single RGB
Images, ECCV 2018

Primary Dataset: ShapeNet

Owner: Kalash

1. Executive Summary

This project aims to build a single-image 3D reconstruction system that takes a 2D RGB product photograph and generates a usable 3D mesh of the object. The system is designed to work without multiple viewpoints, depth sensors, or manual 3D modelling.

The reconstruction model is based on the main idea of Pixel2Mesh: a fixed ellipsoid mesh is progressively deformed using image features and a Graph CNN until it matches the 3D shape of the object. The model will be trained and evaluated using ShapeNet, with Chamfer Distance and F-score used to measure reconstruction quality.

The trained model will be exposed through a FastAPI backend and connected to a React frontend. A user will upload an image, receive a reconstructed mesh, rotate and inspect it in a browser-based 3D viewer, and view a quality report.

2. Problem Statement

Build a model that reconstructs 3D mesh representations from 2D product images.

E-commerce platforms and AR/VR product teams increasingly need 3D representations of physical products for features such as view in your room, 360-degree product spins, virtual try-on, and immersive AR/VR shopping catalogues. However, most product catalogues still contain only flat 2D photographs because:

- 3D scanning hardware such as structured-light scanners, LiDAR rigs, and photogrammetry setups can be expensive and require a dedicated setup.
- Manual 3D modelling by a CAD or 3D artist is slow and does not scale well to catalogues with thousands or millions of products.
- Multi-view photogrammetry needs many photos of the same object from different angles, while most sellers already have only standard product photos.

The gap is therefore between the large amount of existing 2D product photography and the small number of usable 3D assets. This project addresses that gap by learning to convert a single existing product image into an interactive 3D mesh automatically.

3. Objectives

- Build and train a Graph CNN + mesh deformation model for single-image 3D reconstruction.
- Use ShapeNet with the official reconstruction split for training and final evaluation.
- Measure model quality using Chamfer Distance and F-score.
- Expose the trained model through FastAPI REST APIs.
- Build a React frontend with an interactive 3D viewer and quality report.
- Deliver a complete end-to-end flow: image upload -> reconstruction -> 3D view -> quality report.

4. Proposed Solution

The system will have three main parts:

Part	Main Responsibility
Image Feature Extraction	A CNN extracts visual features from the input product image.
3D Reconstruction	A Graph CNN deforms an ellipsoid mesh step by step toward the object shape.
Application Layer	FastAPI serves the model; React and three.js provide upload, 3D viewing, and reporting.

Working flow:

- User uploads one RGB product image through the web interface.
- The React frontend sends the image to the FastAPI backend.
- The backend resizes and normalizes the image and sends it to the trained model.
- The CNN extracts image features.
- The Graph CNN uses these features to move the vertices of an initial ellipsoid mesh.
- The mesh is refined from coarse to fine stages using graph unpooling.
- The final vertices and faces are converted to a standard mesh format such as OBJ or GLB.
- The frontend loads the mesh in three.js and shows the quality report where ground-truth evaluation is available.

5. Why This Approach

Approach	Main Issue	Why this project uses mesh deformation
Voxel based	High memory use and coarse surfaces	Mesh output is directly renderable and more suitable for web and AR/VR.
Point cloud based	No explicit surface connectivity; extra meshing step is needed	Mesh deformation directly produces connected faces.
Mesh deformation	Needs careful training and geometry losses	Matches the Pixel2Mesh research direction and produces a usable mesh directly.

6. Differentiation

- Single-image input: the project is designed around one existing product photograph instead of requiring a new multi-view capture session.
- Measured quality: the model is evaluated on ShapeNet using Chamfer Distance and F-score, rather than relying only on visual judgement.
- End-to-end product: the final output is not only a trained model, but also an API, interactive viewer, and quality report.
- E-commerce focus: the system targets product images and a practical web workflow rather than only a research notebook.

7. Expected Impact and Users

User	Expected Value
E-commerce teams	Generate 3D previews from existing catalogue photos and extend 3D coverage to more products.
AR/VR teams	Receive lightweight mesh outputs that can be rendered in standard 3D pipelines.
Marketplace teams	Use an API-based flow that can be connected to product upload systems.
Shoppers	Get a richer product view through interactive 3D instead of only flat images.

8. Project Scope

8.1 In Scope

- Single-image 3D mesh reconstruction using Graph CNN + mesh deformation trained on ShapeNet.
- Model training and evaluation using Chamfer Distance and F-score.
- FastAPI backend for model inference.
- React frontend with image upload, interactive 3D viewer, and quality report.
- Deployment-ready packaging and documentation.

8.2 Out of Scope for the 3-Month Cycle

- Multi-object scene reconstruction.
- Real-time video or multi-view reconstruction.
- Full photorealistic texture reconstruction as a core deliverable.
- Production-scale billing, multi-tenant infrastructure, or advanced authentication.

9. Research and Data

The grounding paper is Pixel2Mesh: Generating 3D Mesh Models from Single RGB Images (ECCV 2018). The paper uses image features, an initial ellipsoid mesh, Graph CNN based deformation, and coarse-to-fine refinement. This project follows the same high-level idea while building the complete training and application pipeline.

Training data will come from ShapeNet, using the reconstruction data and official split referenced by the Pixel2Mesh workflow. The training pipeline will pair rendered images with corresponding ground-truth 3D geometry.

Data preparation will include dataset organisation, image resizing and normalization, verification of train/test separation, mesh or point-cloud sampling for metric computation, and a validation split created only from the training portion.

10. Technology Stack

Technology	Use in the Project	Why Needed
Python	Training pipeline and backend	Main language for PyTorch, geometry processing, and FastAPI.
PyTorch	Model training and inference	Provides GPU support, automatic differentiation, and custom model building.
PyTorch3D	3D operations and metrics	Provides mesh utilities, differentiable geometry operations, and Chamfer Distance support.
FastAPI	Backend REST API	Simple way to serve the trained model and expose documented endpoints.
React	Frontend	Builds upload, result, and report screens.
JavaScript	Frontend language	Used for React and browser-side 3D integration.
three.js	Interactive 3D viewer	Renders the reconstructed mesh in the browser with rotate/zoom/pan.
trimesh	Mesh processing	Loads, validates, converts, and exports mesh files.

Technology	Use in the Project	Why Needed
Tailwind CSS	UI styling	Keeps the interface clean and consistent.
Axios	Frontend API calls	Handles image upload and requests to FastAPI.
Git/GitHub	Version control	Tracks model, backend, frontend, experiments, and documentation.

11. High Level Design

The project is divided into three connected layers:

- Training layer: ShapeNet -> preprocessing -> CNN + Graph CNN -> losses -> trained checkpoint.
- Backend layer: image validation -> preprocessing -> model inference -> mesh post-processing -> report.
- Frontend layer: image upload -> API request -> 3D viewer -> quality report.

The trained checkpoint produced by the training layer is loaded by the FastAPI backend. The frontend communicates with the backend through REST APIs.

12. Low Level Design

Module	Responsibility
Data loader	Loads images and ground-truth geometry according to the ShapeNet split.
CNN backbone	Extracts image features at useful resolutions.
Graph CNN	Uses mesh connectivity and predicts vertex movements.
Mesh deformation network	Runs coarse-to-fine deformation stages and refinement.
Loss module	Combines shape and mesh regularisation losses.
Training loop	Runs training, validation, checkpointing, and metric logging.
Inference module	Loads the trained model and reconstructs a mesh from a new image.
Mesh post-processing	Cleans and exports the output mesh.
Evaluation module	Computes Chamfer Distance and F-

Module	Responsibility
	score for benchmark evaluation.
React UI	Handles upload, result display, and report presentation.

13. Model Training Pipeline

- Image preprocessing: resize and normalize input images in the same way during training and inference.
- Image features: pass the image through the CNN backbone to obtain visual features.
- Template: start reconstruction from a fixed ellipsoid mesh.
- First deformation stage: the Graph CNN predicts per-vertex coordinate changes using image guidance.
- Graph unpooling: increase mesh resolution before the next stage.
- Further deformation: refine the mesh through additional Graph CNN stages.
- Loss computation: compare the predicted geometry with ground truth and apply mesh regularisation.
- Checkpointing: save the best validation model and keep experiment results for comparison.
- Final evaluation: run the selected checkpoint on the untouched official test split.

14. Losses and Evaluation

Component	Purpose
Chamfer Distance	Measures how close predicted surface points are to the ground-truth surface.
Normal consistency	Encourages correct surface orientation.
Laplacian regularisation	Keeps the mesh smooth and reduces unwanted deformation.
Edge-length regularisation	Helps avoid stretched, collapsed, or degenerate triangles.
F-score	Measures how well predicted surface points match the ground truth within a chosen threshold.

Final benchmark evaluation will use the official ShapeNet test split only after training and tuning are complete. Results should be reported both per category and as an overall average.

15. Backend API

Endpoint	Method	Purpose
/api/v1/health	GET	Check whether the backend and trained model are ready.
/api/v1/reconstruct	POST	Accept one image and return the reconstructed mesh and request ID.
/api/v1/report/{request_id}	GET	Return the quality report for a generated reconstruction.

The backend will validate file type and size, load the trained checkpoint once at startup, run inference, convert the mesh to a usable format, and return structured responses to the frontend.

16. Frontend and User Flow

Screen	Purpose
Upload	Select or drag and drop one product image and start reconstruction.
Reconstruction View	Show the generated 3D mesh with rotate, zoom, and pan controls.
Quality Report	Show Chamfer Distance, F-score, and basic mesh information where evaluation data is available.
Dashboard (Good-to-have)	Show summary charts across reconstruction results and categories.

17. Resources and Direct Links

Resource	Link
Pixel2Mesh paper	https://arxiv.org/abs/1804.01654
Pixel2Mesh code	https://github.com/nywang16/Pixel2Mesh
ShapeNet	https://shapenet.org/
ShapeNet on Kaggle	https://www.kaggle.com/search?q=ShapeNet
PyTorch	https://pytorch.org/
PyTorch3D	https://pytorch3d.org/
FastAPI	https://fastapi.tiangolo.com/
React	https://react.dev/

Resource	Link
three.js	https://threejs.org/
Tailwind CSS	https://tailwindcss.com/
trimesh	https://trimesh.org/

18. Compute and Storage Requirements

- A CUDA-capable GPU is required for practical model training. Early experiments can use available notebook GPU environments before full training runs.
- Storage is required for rendered images, ground-truth geometry, checkpoints, and generated meshes.
- The best validation checkpoint should be saved and versioned so the backend always uses a known model.

19. Implementation Roadmap

Period	Planned Work	Main Output
Month 1	Study Pixel2Mesh and Graph CNN basics; acquire ShapeNet; set up project structure; build baseline model; start FastAPI and React shells.	Working baseline pipeline + dataset ready + basic app structure
Month 2	Train full Graph CNN + mesh deformation model; tune losses; evaluate with Chamfer Distance/F-score; connect inference API; build viewer and report.	Trained model + working API + first end-to-end demo
Month 3	Complete frontend-backend integration; improve UI; run final test split evaluation; add tests/documentation; deploy and prepare final demo.	Complete application + evaluation report + documentation

20. Testing and Quality Checks

- Test image preprocessing and input validation.
- Test mesh conversion and basic mesh validity.
- Test Chamfer Distance and F-score functions on known examples.
- Test API endpoints, including invalid-image and oversized-file cases.

- Check frontend upload, viewer interaction, and report display.
- Run final evaluation only on the official test split.

21. Risks and Mitigation

Risk	Mitigation
Training takes too long or does not converge	Start with a smaller subset and a single deformation stage before scaling to the full model.
Limited GPU access	Use notebook GPUs for early debugging and save checkpoints regularly.
Dataset access or storage issues	Start dataset setup in Month 1 and validate the pipeline on a smaller subset first.
Poor mesh quality	Use geometry regularisation losses and inspect intermediate outputs, not only final scores.
Late frontend-backend integration issues	Define the API contract early and test it before the model is fully finished.

22. Final Deliverables

- Trained single-image 3D reconstruction model.
- Interactive browser-based 3D viewer.
- Mesh quality report with Chamfer Distance and F-score.
- Working React + FastAPI application.
- Training and evaluation results on the official ShapeNet split.
- README with setup, architecture, reproduction steps, and demo instructions.
- Optional dashboard, tests, and basic authentication as time permits.

23. Success Criteria

- The model produces valid 3D meshes from single RGB images.
- The final model is evaluated consistently on the official ShapeNet test split.
- Chamfer Distance and F-score are recorded and reported clearly.
- A user can complete the full workflow from upload to interactive 3D output.
- The project can be run locally and demonstrated end-to-end.

24. Stretch Goal

Add texture mapping to the reconstructed mesh so that the final 3D model is visually closer to the original product photograph and more useful for AR/VR presentations.